CHECKLIST IMPLEMENTATION COCKPIT OBSIDIA X108

Version V1 corrigee - support de pre-build et plan d execution

Document relie au cahier des charges DEV Cockpit Obsidia / X108

CHECKLIST_IMPLEMENTATION_COCKPIT_OBSIDIA_X108_V1_CORRIGEE

Statut du document

Ce document est un support de preparation a l'implementation. Il ne lance aucun build produit. Il transforme le cahier des charges du Cockpit / WorkCenter Obsidia X108 en plan d'execution pour developpeurs, designers, testeurs et copilotes IA.

Le document est volontairement separe du cahier des charges principal. Le cahier des charges explique le projet, les pages et les principes. Cette checklist explique quoi creer, dans quel ordre, pourquoi le creer, quel fichier toucher, quelle route exposer, quelle page relier, quel test ecrire, quel critere de reussite appliquer et ce qui est interdit.

Correction importante sur le Kernel : la mutation directe du Kernel depuis le cockpit est interdite. Cela ne veut pas dire que le Kernel ne peut jamais evoluer. Une evolution du Kernel doit passer par un protocole separe : demande explicite, branche dediee, tests, preuves, audit, freeze, validation humaine, puis merge controle. Le cockpit ne doit jamais devenir un bouton cache qui modifie le noyau souverain.

0. Regle de lecture

Chaque bloc ci-dessous doit etre lu comme une fiche d'execution. Le developpeur doit pouvoir prendre une ligne et savoir immediatement :

- quelle partie du cahier des charges elle applique ;
- quelle page du cockpit elle concerne ;
- quel fichier creer ou modifier ;
- quelle route backend ajouter ;
- quelle donnee afficher ;
- quel test prouve que c'est bon ;
- quel comportement est interdit.

La checklist est organisee en versions progressives V0 a V5. V0 est la preparation sans build. V1 construit la base readonly. V2 ajoute la gouvernance et la validation. V3 ajoute preuve, replay, freeze et education. V4 ajoute sandbox, multimodal, terrain et observabilite. V5 correspond a la cible finale maximale, propre, carree, maintenable et institutionnelle.

1. Principes d'implementation non negociables

1.1 Ce que le cockpit est

Le cockpit est une surface de pilotage, de lecture, de validation, d'audit, d'organisation et d'apprentissage. Il aide l'utilisateur a comprendre le systeme, suivre ses etats, preparer des validations, organiser la memoire, consulter les preuves et planifier les freezes.

1.2 Ce que le cockpit n'est pas

Le cockpit n'est pas un Kernel bis. Le cockpit n'est pas une source de decision souveraine. Le cockpit n'est pas un contournement de X108. Le cockpit n'est pas une interface qui ecrit directement dans Graphiti. Le cockpit n'est pas une interface qui promeut une memoire en canon sans validation humaine. Le cockpit n'est pas une interface qui transforme un savoir candidate en doctrine sans trace, sans preuve et sans approbation.

1.3 Regle corrigee sur la mutation Kernel

Formulation a utiliser :

- Interdit : mutation directe du Kernel depuis l'UI, depuis une route WorkCenter, depuis Brody, depuis Graphiti, depuis une memoire candidate ou depuis un bouton de cockpit.
- Autorise : evolution du Kernel via protocole de changement controle.
- Protocole obligatoire : demande de changement, dossier d'impact, branche dediee, tests unitaires, tests d'integration, tests adversariaux si necessaires, preuves ou verification associee, rapport de risque, freeze pre-merge, validation humaine, merge controle.

Cette nuance doit apparaitre partout. On ne dit pas : "Kernel intouchable pour toujours". On dit : "Kernel non mutable directement par le cockpit".

2. Matrice globale des versions

Version	Nom	Role	Pages concernees	Build produit autorise	Sortie attendue
V0	Pre-build cadrage	Transformer le cahier des charges en plan executable	Toutes les pages	Non	registries spec, wireframes textuels, backlog, criteres
V1	Readonly Foundation	Lire le backend et afficher l'etat reel	Dashboard, Authority Graph, Server Advisory, Resource Library, Session Ledger readonly	Oui, minimal readonly	cockpit consultatif fiable
V2	Governance Core	Ajouter validation humaine, process validation et memoire candidate	Memory Governance, Human Validation, Process Validation, Session Ledger	Oui	workflow de gouvernance sans auto-promotion
V3	Proof Freeze Education	Ajouter Proof Lab, Freeze Factory advisory et Brody Education	Proof & Replay, Freeze Factory, Brody Education	Oui	preuve, replay, education, freeze candidate
V4	Terrain Sandbox Multimodal	Ajouter branches terrain, sandbox, multimodal, observabilite avancee	Terrain Branching, Coding Sandbox, Multimodal Workbench, Server Advisory avance	Oui	cockpit operable mais gouverne
V5	Maximal Final	Durcir, industrialiser, auditer et stabiliser	Toutes les pages	Oui, apres V1-V4	WorkCenter final propre, maintenable, audit-ready

3. V0 - Pre-build cadrage obligatoire

Objectif : ne pas coder dans le flou. V0 produit les artefacts qui empechent les devs d'improviser.

3.1 Checklist V0 - Decision de perimetre

[ ] Confirmer que le cockpit V1 est readonly first.

[ ] Confirmer que les actions d'ecriture seront introduites seulement comme demandes, pas comme mutations directes.

[ ] Confirmer que la mutation directe Kernel depuis cockpit est interdite.

[ ] Confirmer que l'evolution Kernel passe par le Kernel Change Protocol.

[ ] Confirmer que Graphiti reste readonly tant qu'une procedure explicite de write governance n'existe pas.

[ ] Confirmer que Brody Education reste non souverain.

[ ] Confirmer que la memoire canonique exige validation humaine.

[ ] Confirmer que la memoire preuve exige artefact verifiable.

[ ] Confirmer que le freeze exige manifest, evidence et smoke.

[ ] Confirmer que le cockpit doit afficher les limites et les statuts, pas masquer les trous.

3.2 Checklist V0 - Cartographie existant vs a creer

[ ] Lister les services existants : API FastAPI 8000, Kernel 3001, Graphiti 8011, Brody 8012, UI 5173, Neo4j, proofs, Sigma, CIC, domaines.

[ ] Lister les routes existantes disponibles dans OpenAPI.

[ ] Lister les routes manquantes WorkCenter.

[ ] Lister les registries a creer.

[ ] Lister les pages UI a creer.

[ ] Lister les workflows a implementer.

[ ] Lister les tests attendus par page.

[ ] Lister les fichiers existants que le dev ne doit pas modifier.

[ ] Lister les fichiers nouveaux a creer dans une zone WorkCenter dediee.

[ ] Lister les claims interdits.

3.3 Fichiers V0 a produire

- docs/workcenter/CHECKLIST_IMPLEMENTATION_COCKPIT_OBSIDIA_X108_V1.md
- docs/workcenter/WORKCENTER_SCOPE_V1.md
- docs/workcenter/WORKCENTER_PAGES_MAP.md
- docs/workcenter/WORKCENTER_ROUTES_PLAN.md
- docs/workcenter/WORKCENTER_STATUS_DICTIONARY.md
- docs/workcenter/WORKCENTER_FORBIDDEN_ACTIONS.md
- docs/workcenter/WORKCENTER_KERNEL_CHANGE_PROTOCOL.md
- docs/workcenter/WORKCENTER_ACCEPTANCE_CRITERIA.md
- docs/workcenter/WORKCENTER_BACKLOG_V1_TO_V5.md

3.4 Criteres de reussite V0

[ ] Un dev peut expliquer les pages V1 sans relire tout le cahier des charges.

[ ] Un dev peut identifier les fichiers a creer.

[ ] Un dev peut identifier les routes a exposer.

[ ] Un dev peut identifier les tests a ecrire.

[ ] Un dev sait que le cockpit ne decide pas.

[ ] Un dev sait que le Kernel peut evoluer uniquement via protocole dedie.

[ ] Aucune ligne de code produit n'a ete modifiee avant validation V0.

4. Socle technique recommande

Le socle minimal est acceptable pour demarrer, mais la cible finale doit rester maximale. Il faut donc construire avec une architecture qui peut grandir sans etre refaite.

4.1 Frontend

- Application existante ou nouvelle zone dans apps/obsidia-workbench.
- React avec TypeScript.
- Router par pages WorkCenter.
- Sidebar fixe.
- Header d'environnement.
- Systeme de badges statut.
- Tables filtrables.
- Timeline reusable.
- Graph authority reusable.
- Cards metrics reusable.
- Drawer de details reutilisable.
- Error boundary par page.
- Empty states explicites.
- Loading states explicites.

4.2 Backend

- Module dedie WorkCenter dans l'API existante.
- Routes readonly en GET pour V1.
- POST limites a des demandes ou dry-run en V1/V2.
- Services de lecture registries.
- Services de lecture fichiers evidence.
- Schemas Pydantic dedies.
- Aucun appel mutation Kernel depuis WorkCenter.
- Aucun write Graphiti direct.
- Aucun commit direct depuis WorkCenter.

4.3 Stockage V1

- JSON registries versionnes.
- Snapshots OpenAPI.
- Rapports d'audit existants.
- Repertoires local_freezes.
- Rapports proofs.
- Session summaries.
- Ressources documentaires.

4.4 Cible maximale V5

- Base relationnelle pour sessions, validations, ressources, tickets et workflows.
- Event log append-only.
- Signature/hash pour actions sensibles.
- Workers asynchrones pour proofs, freeze, replays, scans.
- Observabilite OTEL.
- Gestion multi-utilisateur.
- Permissions par role.
- Journal d'audit exportable.
- Integration cockpit/workbench sans melange d'autorite.

5. Arborescence cible

```
apps/
  obsidia_api/
    routes/
      workcenter.py
    services/
      workcenter_registry_service.py
      workcenter_status_service.py
      workcenter_memory_service.py
      workcenter_validation_service.py
      workcenter_proof_service.py
      workcenter_freeze_service.py
      workcenter_resource_service.py
      workcenter_server_advisory_service.py
      workcenter_kernel_change_service.py
    schemas/
      workcenter_status.py
      workcenter_component.py
      workcenter_memory.py
      workcenter_validation.py
      workcenter_proof.py
      workcenter_freeze.py
      workcenter_resource.py
      workcenter_server.py
      workcenter_kernel_change.py

apps/
  obsidia-workbench/
    src/
      pages/
        workcenter/
          DashboardPage.tsx
          AuthorityGraphPage.tsx
          MemoryGovernancePage.tsx
          HumanValidationPage.tsx
          ProcessValidationPage.tsx
          BrodyEducationPage.tsx
          TerrainBranchingPage.tsx
          ProofReplayLabPage.tsx
          ResourceLibraryPage.tsx
          SessionLedgerPage.tsx
          FreezeFactoryPage.tsx
          CodingSandboxPage.tsx
          ServerAdvisoryPage.tsx
          MultimodalWorkbenchPage.tsx
          GovernancePoliciesPage.tsx
      components/
        workcenter/
          StatusBadge.tsx
          AuthorityGraph.tsx
          MetricCard.tsx
          RegistryTable.tsx
          EvidenceDrawer.tsx
          Timeline.tsx
          ValidationQueue.tsx
          ProofStatusPanel.tsx
          FreezeCandidatePanel.tsx
          ForbiddenActionBanner.tsx
      api/
        workcenterClient.ts

workcenter/
  registries/
    components_registry.json
    servers_registry.json
    routes_registry.json
```

```
    domains_registry.json
    memory_registry.json
    proofs_registry.json
    freezes_registry.json
    resources_registry.json
    brody_training_registry.json
    validation_registry.json
    forbidden_actions_registry.json
    kernel_change_registry.json
  schemas/
    registry_schema.json
    status_schema.json
    validation_schema.json
  reports/
    WORKCENTER_BASELINE_REPORT.md
    WORKCENTER_ACCEPTANCE_REPORT.md
```

6. Matrice page par page liee au cahier des charges

Chaque page ci-dessous est reliee au cahier des charges. Le developpeur doit garder le cahier des charges ouvert pour la narration et utiliser cette section pour l'execution.

6.1 Page - Dashboard global

Lien cahier des charges : CDC - Page Dashboard global.

Pourquoi creer cette page : Donner en 30 secondes l'etat reel du systeme Obsidia / X108.

Donnees a afficher :

- ☐ statut API 8000
- ☐ statut Kernel 3001
- ☐ statut Brody
- ☐ statut Graphiti
- ☐ statut UI 5173
- ☐ statut Neo4j
- ☐ dernier freeze
- ☐ dernier test
- ☐ dernieres decisions
- ☐ risques ouverts

Fichiers frontend a creer :

- ☐ apps/obsidia-workbench/src/pages/workcenter/DashboardglobalPage.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/DashboardglobalPanel.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/DashboardglobalFilters.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/DashboardglobalDetailDrawer.tsx

Routes backend a utiliser ou creer :

- ☐ GET /api/workcenter/status
- ☐ GET /api/workcenter/metrics
- ☐ GET /api/workcenter/servers
- ☐ GET /api/workcenter/freezes

Tests a ecrire :

- ☐ tests/test_workcenter_status.py
- ☐ tests/test_workcenter_metrics_readonly.py

Critere de reussite :

- ☐ La page Dashboard global charge sans erreur quand l'API repond.
- ☐ La page Dashboard global affiche un etat vide explicite si aucune donnee n'existe.
- ☐ La page Dashboard global affiche un etat erreur explicite si la route backend echoue.
- ☐ La page Dashboard global n'invente aucun statut non present dans les registries.
- ☐ La page Dashboard global n'effectue aucune action interdite.

Interdits specifique page :

- ☐ Pas de decision souveraine dans le frontend.
- ☐ Pas de mutation directe du Kernel.
- ☐ Pas d'ecriture Graphiti directe.
- ☐ Pas de promotion canon sans validation humaine.
- ☐ Pas de claim confirme sans source runtime, test, freeze ou code.

6.2 Page - Authority Graph

Lien cahier des charges : CDC - Page Authority Graph.

Pourquoi creer cette page : Montrer qui a autorite, qui aide, qui memorise, qui route, qui verifie.

Donnees a afficher :

- ☐ Kernel X108 souverain
- ☐ FastAPI bridge
- ☐ Brody education readonly
- ☐ Graphiti memory readonly
- ☐ CIC context readonly
- ☐ Sigma aggregation
- ☐ Proof stack
- ☐ Human validation

Fichiers frontend a creer :

- ☐ apps/obsidia-workbench/src/pages/workcenter/AuthorityGraphPage.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/AuthorityGraphPanel.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/AuthorityGraphFilters.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/AuthorityGraphDetailDrawer.tsx

Routes backend a utiliser ou creer :

- ☐ GET /api/workcenter/authority
- ☐ GET /api/workcenter/components

Tests a ecrire :

- ☐ tests/test_authority_graph_boundaries.py

Critere de reussite :

- ☐ La page Authority Graph charge sans erreur quand l'API repond.
- ☐ La page Authority Graph affiche un etat vide explicite si aucune donnee n'existe.
- ☐ La page Authority Graph affiche un etat erreur explicite si la route backend echoue.
- ☐ La page Authority Graph n'invente aucun statut non present dans les registries.
- ☐ La page Authority Graph n'effectue aucune action interdite.

Interdits specifique page :

- ☐ Pas de decision souveraine dans le frontend.
- ☐ Pas de mutation directe du Kernel.
- ☐ Pas d'ecriture Graphiti directe.

- ☐ Pas de promotion canon sans validation humaine.
- ☐ Pas de claim confirme sans source runtime, test, freeze ou code.

6.3 Page - Memory Governance Center

Lien cahier des charges : CDC - Page Memory Governance.

Pourquoi creer cette page : Gerer le cycle de vie des memoires sans promotion automatique.

Donnees a afficher :

- ☐ RAW_CAPTURE
- ☐ SESSION_SCRATCH
- ☐ CANDIDATE
- ☐ NEEDS_HUMAN_REVIEW
- ☐ NEEDS_PROCESS_REVIEW
- ☐ PROCESS_VALIDATED
- ☐ HUMAN_VALIDATED
- ☐ PROMOTED_TO_CANON
- ☐ PROMOTED_TO_TRAINING
- ☐ PROMOTED_TO_OPERATIONAL
- ☐ PROMOTED_TO_DOMAIN
- ☐ PROMOTED_TO_PROOF
- ☐ REJECTED
- ☐ SUPERSEDED
- ☐ DEPRECATED
- ☐ FORBIDDEN_SECRET

Fichiers frontend a creer :

- ☐ apps/obsidia-workbench/src/pages/workcenter/MemoryGovernanceCenterPage.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/MemoryGovernanceCenterPanel.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/MemoryGovernanceCenterFilters.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/MemoryGovernanceCenterDetailDrawer.tsx

Routes backend a utiliser ou creer :

- ☐ GET /api/workcenter/memory/candidates
- ☐ GET /api/workcenter/memory/promoted
- ☐ POST /api/workcenter/memory/candidate
- ☐ POST /api/workcenter/validation/request

Tests a ecrire :

- ☐ tests/test_memory_governance_statuses.py
- ☐ tests/test_memory_no_auto_promotion.py

Critere de reussite :

- ☐ La page Memory Governance Center charge sans erreur quand l'API repond.
- ☐ La page Memory Governance Center affiche un etat vide explicite si aucune donnee n'existe.
- ☐ La page Memory Governance Center affiche un etat erreur explicite si la route backend echoue.
- ☐ La page Memory Governance Center n'invente aucun statut non present dans les registries.
- ☐ La page Memory Governance Center n'effectue aucune action interdite.

Interdits specifique page :

- ☐ Pas de decision souveraine dans le frontend.
- ☐ Pas de mutation directe du Kernel.

- ☐ Pas d'ecriture Graphiti directe.
- ☐ Pas de promotion canon sans validation humaine.
- ☐ Pas de claim confirme sans source runtime, test, freeze ou code.

6.4 Page - Human Validation Console

Lien cahier des charges : CDC - Page Human Validation.

Pourquoi creer cette page : Donner a l'humain le controle sur canon, doctrine, savoir durable, promotion sensible et freeze.

Donnees a afficher :

- ☐ pending validations
- ☐ memory candidates
- ☐ proof claims
- ☐ freeze requests
- ☐ patch proposals
- ☐ canon promotions
- ☐ domain promotions

Fichiers frontend a creer :

- ☐ apps/obsidia-workbench/src/pages/workcenter/HumanValidationConsolePage.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/HumanValidationConsolePanel.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/HumanValidationConsoleFilters.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/HumanValidationConsoleDetailDrawer.tsx

Routes backend a utiliser ou creer :

- ☐ GET /api/workcenter/validation/human
- ☐ POST /api/workcenter/validation/human/approve
- ☐ POST /api/workcenter/validation/human/reject

Tests a ecrire :

- ☐ tests/test_human_validation_required_for_canon.py

Critere de reussite :

- ☐ La page Human Validation Console charge sans erreur quand l'API repond.
- ☐ La page Human Validation Console affiche un etat vide explicite si aucune donnee n'existe.
- ☐ La page Human Validation Console affiche un etat erreur explicite si la route backend echoue.
- ☐ La page Human Validation Console n'invente aucun statut non present dans les registries.
- ☐ La page Human Validation Console n'effectue aucune action interdite.

Interdits specifique page :

- ☐ Pas de decision souveraine dans le frontend.
- ☐ Pas de mutation directe du Kernel.
- ☐ Pas d'ecriture Graphiti directe.
- ☐ Pas de promotion canon sans validation humaine.
- ☐ Pas de claim confirme sans source runtime, test, freeze ou code.

6.5 Page - Process Validation Console

Lien cahier des charges : CDC - Page Process Validation.

Pourquoi creer cette page : Confirmer par preuve process ce qui peut etre valide par test, runtime, freeze, code ou audit.

Donnees a afficher :

- ☐ CONFIRMED_BY_RUNTIME
- ☐ CONFIRMED_BY_TEST
- ☐ CONFIRMED_BY_FREEZE
- ☐ CONFIRMED_BY_CODE
- ☐ PRESENT_NOT_CONNECTED
- ☐ DOCUMENTED_ONLY
- ☐ LEGACY
- ☐ BLOCKED
- ☐ FAILED
- ☐ UNKNOWN

Fichiers frontend a creer :

- ☐ apps/obsidia-workbench/src/pages/workcenter/ProcessValidationConsolePage.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/ProcessValidationConsolePanel.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/ProcessValidationConsoleFilters.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/ProcessValidationConsoleDetailDrawer.tsx

Routes backend a utiliser ou creer :

- ☐ GET /api/workcenter/validation/process
- ☐ POST /api/workcenter/validation/process/request

Tests a ecrire :

- ☐ tests/test_process_validation_statuses.py

Critere de reussite :

- ☐ La page Process Validation Console charge sans erreur quand l'API repond.
- ☐ La page Process Validation Console affiche un etat vide explicite si aucune donnee n'existe.
- ☐ La page Process Validation Console affiche un etat erreur explicite si la route backend echoue.
- ☐ La page Process Validation Console n'invente aucun statut non present dans les registries.
- ☐ La page Process Validation Console n'effectue aucune action interdite.

Interdits specifique page :

- ☐ Pas de decision souveraine dans le frontend.
- ☐ Pas de mutation directe du Kernel.
- ☐ Pas d'ecriture Graphiti directe.
- ☐ Pas de promotion canon sans validation humaine.
- ☐ Pas de claim confirme sans source runtime, test, freeze ou code.

6.6 Page - Brody Education Station

Lien cahier des charges : CDC - Page Brody Education.

Pourquoi creer cette page : Transformer Brody en station d'apprentissage non souveraine avec modules T1 a T20.

Donnees a afficher :

- ☐ modules T1 a T20
- ☐ lesson status
- ☐ quiz score
- ☐ exercise result
- ☐ forbidden actions
- ☐ training candidates

Fichiers frontend a creer :

- ☐ apps/obsidia-workbench/src/pages/workcenter/BrodyEducationStationPage.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/BrodyEducationStationPanel.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/BrodyEducationStationFilters.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/BrodyEducationStationDetailDrawer.tsx

Routes backend a utiliser ou creer :

- ☐ GET /api/workcenter/training/modules
- ☐ GET /api/workcenter/training/modules/{module_id}
- ☐ POST /api/workcenter/training/answer

Tests a ecrire :

- ☐ tests/test_brody_training_readonly.py
- ☐ tests/test_brody_no_act_no_memory_write.py

Critere de reussite :

- ☐ La page Brody Education Station charge sans erreur quand l'API repond.
- ☐ La page Brody Education Station affiche un etat vide explicite si aucune donnee n'existe.
- ☐ La page Brody Education Station affiche un etat erreur explicite si la route backend echoue.
- ☐ La page Brody Education Station n'invente aucun statut non present dans les registries.
- ☐ La page Brody Education Station n'effectue aucune action interdite.

Interdits specifique page :

- ☐ Pas de decision souveraine dans le frontend.
- ☐ Pas de mutation directe du Kernel.
- ☐ Pas d'ecriture Graphiti directe.
- ☐ Pas de promotion canon sans validation humaine.
- ☐ Pas de claim confirme sans source runtime, test, freeze ou code.

6.7 Page - Terrain Branching Center

Lien cahier des charges : CDC - Page Terrain Branching.

Pourquoi creer cette page : Organiser les domaines et les branches d'experimentation sans action live non gouvernee.

Donnees a afficher :

- ☐ Bank
- ☐ Trading
- ☐ GPS / Aviation / Defense
- ☐ Ecom
- ☐ CIC
- ☐ NCP readonly
- ☐ Scraping readonly
- ☐ future domains

Fichiers frontend a creer :

- ☐ apps/obsidia-workbench/src/pages/workcenter/TerrainBranchingCenterPage.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/TerrainBranchingCenterPanel.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/TerrainBranchingCenterFilters.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/TerrainBranchingCenterDetailDrawer.tsx

Routes backend a utiliser ou creer :

- ☐ GET /api/workcenter/terrain/domains
- ☐ GET /api/workcenter/terrain/branches
- ☐ POST /api/workcenter/terrain/branch/request

Tests a ecrire :

- ☐ tests/test_terrain_branch_readonly_first.py

Critere de reussite :

- ☐ La page Terrain Branching Center charge sans erreur quand l'API repond.
- ☐ La page Terrain Branching Center affiche un etat vide explicite si aucune donnee n'existe.
- ☐ La page Terrain Branching Center affiche un etat erreur explicite si la route backend echoue.
- ☐ La page Terrain Branching Center n'invente aucun statut non present dans les registries.
- ☐ La page Terrain Branching Center n'effectue aucune action interdite.

Interdits specifique page :

- ☐ Pas de decision souveraine dans le frontend.
- ☐ Pas de mutation directe du Kernel.
- ☐ Pas d'ecriture Graphiti directe.
- ☐ Pas de promotion canon sans validation humaine.
- ☐ Pas de claim confirme sans source runtime, test, freeze ou code.

6.8 Page - Proof and Replay Lab

Lien cahier des charges : CDC - Page Proof & Replay.

Pourquoi creer cette page : Afficher, lancer sous controle et relire les preuves et replays.

Donnees a afficher :

- ☐ Lean
- ☐ 57 theorems
- ☐ TLA
- ☐ verify_all.py
- ☐ verify_decision.py
- ☐ verify_merkle.py
- ☐ Merkle
- ☐ RFC3161
- ☐ proofkit
- ☐ decision replay

Fichiers frontend a creer :

- ☐ apps/obsidia-workbench/src/pages/workcenter/ProofandReplayLabPage.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/ProofandReplayLabPanel.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/ProofandReplayLabFilters.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/ProofandReplayLabDetailDrawer.tsx

Routes backend a utiliser ou creer :

- ☐ GET /api/workcenter/proofs
- ☐ POST /api/workcenter/proofs/run/request
- ☐ GET /api/workcenter/replay/{decision_id}

Tests a ecrire :

- ☐ tests/test_proof_lab_no_false_confirmed_claim.py
- ☐ tests/test_replay_readonly.py

Critere de reussite :

- ☐ La page Proof and Replay Lab charge sans erreur quand l'API repond.
- ☐ La page Proof and Replay Lab affiche un etat vide explicite si aucune donnee n'existe.
- ☐ La page Proof and Replay Lab affiche un etat erreur explicite si la route backend echoue.
- ☐ La page Proof and Replay Lab n'invente aucun statut non present dans les registries.
- ☐ La page Proof and Replay Lab n'effectue aucune action interdite.

Interdits specifique page :

- ☐ Pas de decision souveraine dans le frontend.
- ☐ Pas de mutation directe du Kernel.
- ☐ Pas d'ecriture Graphiti directe.
- ☐ Pas de promotion canon sans validation humaine.
- ☐ Pas de claim confirme sans source runtime, test, freeze ou code.

6.9 Page - Resource Library

Lien cahier des charges : CDC - Page Resource Library.

Pourquoi creer cette page : Eviter la confusion entre fichiers actifs, anciens, freeze, docs, proof outputs et ressources de formation.

Donnees a afficher :

- ☐ README
- ☐ glossaire
- ☐ audits
- ☐ freezes
- ☐ prompts
- ☐ schemas
- ☐ payloads
- ☐ scripts
- ☐ proof outputs
- ☐ training modules
- ☐ operator guides
- ☐ error catalog
- ☐ diagrams
- ☐ multimodal extracts

Fichiers frontend a creer :

- ☐ apps/obsidia-workbench/src/pages/workcenter/ResourceLibraryPage.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/ResourceLibraryPanel.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/ResourceLibraryFilters.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/ResourceLibraryDetailDrawer.tsx

Routes backend a utiliser ou creer :

- ☐ GET /api/workcenter/resources
- ☐ GET /api/workcenter/resources/{resource_id}

Tests a ecrire :

- ☐ tests/test_resource_library_truth_levels.py

Critere de reussite :

- ☐ La page Resource Library charge sans erreur quand l'API repond.

- ☐ La page Resource Library affiche un etat vide explicite si aucune donnee n'existe.
- ☐ La page Resource Library affiche un etat erreur explicite si la route backend echoue.
- ☐ La page Resource Library n'invente aucun statut non present dans les registries.
- ☐ La page Resource Library n'effectue aucune action interdite.

Interdits specifique page :

- ☐ Pas de decision souveraine dans le frontend.
- ☐ Pas de mutation directe du Kernel.
- ☐ Pas d'ecriture Graphiti directe.
- ☐ Pas de promotion canon sans validation humaine.
- ☐ Pas de claim confirme sans source runtime, test, freeze ou code.

6.10 Page - Session Ledger

Lien cahier des charges : CDC - Page Session Ledger.

Pourquoi creer cette page : Fermer proprement les sessions et separer recap, actions, tests, risques, fichiers touches et prochaines commandes.

Donnees a afficher :

- ☐ SESSION_RECAP.md
- ☐ MEMORY_CANDIDATES.json
- ☐ CLAIMS_TO_VALIDATE.md
- ☐ ACTIONS_DECIDED.md
- ☐ TESTS_RUN.md
- ☐ FILES_TOUCHED.md
- ☐ RISKS_AND_BLOCKERS.md
- ☐ NEXT_COMMANDS.md

Fichiers frontend a creer :

- ☐ apps/obsidia-workbench/src/pages/workcenter/SessionLedgerPage.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/SessionLedgerPanel.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/SessionLedgerFilters.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/SessionLedgerDetailDrawer.tsx

Routes backend a utiliser ou creer :

- ☐ GET /api/workcenter/session/current
- ☐ GET /api/workcenter/sessions
- ☐ POST /api/workcenter/session/close/dry-run

Tests a ecrire :

- ☐ tests/test_session_close_dry_run_outputs.py

Critere de reussite :

- ☐ La page Session Ledger charge sans erreur quand l'API repond.
- ☐ La page Session Ledger affiche un etat vide explicite si aucune donnee n'existe.
- ☐ La page Session Ledger affiche un etat erreur explicite si la route backend echoue.
- ☐ La page Session Ledger n'invente aucun statut non present dans les registries.
- ☐ La page Session Ledger n'effectue aucune action interdite.

Interdits specifique page :

- ☐ Pas de decision souveraine dans le frontend.
- ☐ Pas de mutation directe du Kernel.

- ☐ Pas d'ecriture Graphiti directe.
- ☐ Pas de promotion canon sans validation humaine.
- ☐ Pas de claim confirme sans source runtime, test, freeze ou code.

6.11 Page - Freeze Factory

Lien cahier des charges : CDC - Page Freeze Factory.

Pourquoi creer cette page : Creer des freezes auditables avec evidence, manifest et smoke, sans freeze automatique non valide.

Donnees a afficher :

- ☐ LOCAL_FREEZE
- ☐ RUNTIME_FREEZE
- ☐ PROOF_FREEZE
- ☐ BRODY_FREEZE
- ☐ GRAPHITI_FREEZE
- ☐ DOMAIN_FREEZE
- ☐ SESSION_FREEZE
- ☐ WORKCENTER_FREEZE

Fichiers frontend a creer :

- ☐ apps/obsidia-workbench/src/pages/workcenter/FreezeFactoryPage.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/FreezeFactoryPanel.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/FreezeFactoryFilters.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/FreezeFactoryDetailDrawer.tsx

Routes backend a utiliser ou creer :

- ☐ GET /api/workcenter/freezes
- ☐ POST /api/workcenter/freezes/candidate
- ☐ POST /api/workcenter/freezes/run/request

Tests a ecrire :

- ☐ tests/test_freeze_requires_manifest_and_validation.py

Critere de reussite :

- ☐ La page Freeze Factory charge sans erreur quand l'API repond.
- ☐ La page Freeze Factory affiche un etat vide explicite si aucune donnee n'existe.
- ☐ La page Freeze Factory affiche un etat erreur explicite si la route backend echoue.
- ☐ La page Freeze Factory n'invente aucun statut non present dans les registries.
- ☐ La page Freeze Factory n'effectue aucune action interdite.

Interdits specifique page :

- ☐ Pas de decision souveraine dans le frontend.
- ☐ Pas de mutation directe du Kernel.
- ☐ Pas d'ecriture Graphiti directe.
- ☐ Pas de promotion canon sans validation humaine.
- ☐ Pas de claim confirme sans source runtime, test, freeze ou code.

6.12 Page - Coding Sandbox

Lien cahier des charges : CDC - Page Coding Sandbox.

Pourquoi creer cette page : Lire, inspecter, proposer et tester sans commit direct ni modification sensible non validee.

Donnees a afficher :

- ☐ READ_ONLY_LEARNING
- ☐ TEST_ONLY
- ☐ PATCH_PROPOSAL
- ☐ LOCAL_SANDBOX_WRITE
- ☐ HUMAN_APPROVED_WRITE
- ☐ KERNEL_CHANGE_PROTOCOL

Fichiers frontend a creer :

- ☐ apps/obsidia-workbench/src/pages/workcenter/CodingSandboxPage.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/CodingSandboxPanel.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/CodingSandboxFilters.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/CodingSandboxDetailDrawer.tsx

Routes backend a utiliser ou creer :

- ☐ GET /api/workcenter/sandbox/status
- ☐ POST /api/workcenter/sandbox/patch/propose
- ☐ POST /api/workcenter/sandbox/test/request

Tests a ecrire :

- ☐ tests/test_sandbox_no_direct_commit.py
- ☐ tests/test_sandbox_kernel_change_protocol.py

Critere de reussite :

- ☐ La page Coding Sandbox charge sans erreur quand l'API repond.
- ☐ La page Coding Sandbox affiche un etat vide explicite si aucune donnee n'existe.
- ☐ La page Coding Sandbox affiche un etat erreur explicite si la route backend echoue.
- ☐ La page Coding Sandbox n'invente aucun statut non present dans les registries.
- ☐ La page Coding Sandbox n'effectue aucune action interdite.

Interdits specifique page :

- ☐ Pas de decision souveraine dans le frontend.
- ☐ Pas de mutation directe du Kernel.
- ☐ Pas d'ecriture Graphiti directe.
- ☐ Pas de promotion canon sans validation humaine.
- ☐ Pas de claim confirme sans source runtime, test, freeze ou code.

6.13 Page - Server Control Advisory

Lien cahier des charges : CDC - Page Server Advisory.

Pourquoi creer cette page : Surveiller les ports et proposer des commandes sans kill, restart ou mutation directe.

Donnees a afficher :

- ☐ ports
- ☐ process
- ☐ healthcheck
- ☐ logs
- ☐ commands
- ☐ last_probe

Fichiers frontend a creer :

- ☐ apps/obsidia-workbench/src/pages/workcenter/ServerControlAdvisoryPage.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/ServerControlAdvisoryPanel.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/ServerControlAdvisoryFilters.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/ServerControlAdvisoryDetailDrawer.tsx

Routes backend a utiliser ou creer :

- ☐ GET /api/workcenter/servers
- ☐ GET /api/workcenter/servers/commands/advisory

Tests a ecrire :

- ☐ tests/test_server_advisory_no_direct_restart.py

Critere de reussite :

- ☐ La page Server Control Advisory charge sans erreur quand l'API repond.
- ☐ La page Server Control Advisory affiche un etat vide explicite si aucune donnee n'existe.
- ☐ La page Server Control Advisory affiche un etat erreur explicite si la route backend echoue.
- ☐ La page Server Control Advisory n'invente aucun statut non present dans les registries.
- ☐ La page Server Control Advisory n'effectue aucune action interdite.

Interdits specifique page :

- ☐ Pas de decision souveraine dans le frontend.
- ☐ Pas de mutation directe du Kernel.
- ☐ Pas d'ecriture Graphiti directe.
- ☐ Pas de promotion canon sans validation humaine.
- ☐ Pas de claim confirme sans source runtime, test, freeze ou code.

6.14 Page - Multimodal Workbench

Lien cahier des charges : CDC - Page Multimodal Workbench.

Pourquoi creer cette page : Classer images, audio, video, PDF, captures et logs en ressources candidates sans decision ni ecriture durable directe.

Donnees a afficher :

- ☐ image
- ☐ audio
- ☐ video
- ☐ PDF
- ☐ captures ecran
- ☐ logs
- ☐ schemas
- ☐ slides
- ☐ terminal

Fichiers frontend a creer :

- ☐ apps/obsidia-workbench/src/pages/workcenter/MultimodalWorkbenchPage.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/MultimodalWorkbenchPanel.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/MultimodalWorkbenchFilters.tsx
- ☐ apps/obsidia-workbench/src/components/workcenter/MultimodalWorkbenchDetailDrawer.tsx

Routes backend a utiliser ou creer :

- ☐ GET /api/workcenter/multimodal/resources
- ☐ POST /api/workcenter/multimodal/candidate

Tests a ecrire :

- ☐ tests/test_multimodal_candidate_no_auto_memory.py

Critere de reussite :

- ☐ La page Multimodal Workbench charge sans erreur quand l'API repond.
- ☐ La page Multimodal Workbench affiche un etat vide explicite si aucune donnee n'existe.
- ☐ La page Multimodal Workbench affiche un etat erreur explicite si la route backend echoue.
- ☐ La page Multimodal Workbench n'invente aucun statut non present dans les registries.
- ☐ La page Multimodal Workbench n'effectue aucune action interdite.

Interdits specifique page :

- ☐ Pas de decision souveraine dans le frontend.
- ☐ Pas de mutation directe du Kernel.
- ☐ Pas d'ecriture Graphiti directe.
- ☐ Pas de promotion canon sans validation humaine.
- ☐ Pas de claim confirme sans source runtime, test, freeze ou code.

7.1 V1 - Readonly Foundation

Role : Construire le cockpit de lecture fiable avant toute action.

Pages concernees :

- ☐ Dashboard global
- ☐ Authority Graph
- ☐ Server Control Advisory
- ☐ Resource Library
- ☐ Session Ledger
- ☐ Proof and Replay Lab

Ordre d'execution :

1. ☐ Creer les registries readonly
   - • Fichier ou zone : `workcenter/registries/*.json`
   - • Pourquoi : Avoir une source de verite cockpit avant UI
   - • Critere : test PASS, UI charge, aucune action interdite.
2. ☐ Creer le loader backend registry
   - • Fichier ou zone : `apps/obsidia_api/services/workcenter_registry_service.py`
   - • Pourquoi : Lire les donnees sans hardcode UI
   - • Critere : test PASS, UI charge, aucune action interdite.
3. ☐ Creer routes status et metrics
   - • Fichier ou zone : `apps/obsidia_api/routes/workcenter.py`
   - • Pourquoi : Alimenter Dashboard
   - • Critere : test PASS, UI charge, aucune action interdite.
4. ☐ Creer UI Dashboard

- Fichier ou zone : `DashboardPage.tsx`
- Pourquoi : Verifier la chaine API -> UI
- Critere : test PASS, UI charge, aucune action interdite.

5. ☐ Creer Authority Graph
   - Fichier ou zone : `AuthorityGraphPage.tsx`
   - Pourquoi : Afficher autorite et non-souverainete
   - Critere : test PASS, UI charge, aucune action interdite.
6. ☐ Creer Server Advisory readonly
   - Fichier ou zone : `ServerAdvisoryPage.tsx`
   - Pourquoi : Surveiller sans agir
   - Critere : test PASS, UI charge, aucune action interdite.
7. ☐ Creer Resource Library base
   - Fichier ou zone : `ResourceLibraryPage.tsx`
   - Pourquoi : Centraliser les ressources
   - Critere : test PASS, UI charge, aucune action interdite.

Gate de sortie :

- ☐ Toutes les pages V1 affichent leurs donnees ou un etat vide clair.
- ☐ Tous les tests V1 passent.
- ☐ Le rapport d'acceptance V1 est ecrit.
- ☐ Aucune mutation directe du Kernel, de Graphiti ou du canon n'est possible depuis V1.

7.2 V2 - Governance Core

Role : Introduire la gouvernance memoire et validation sans automatisme dangereux.

Pages concernees :

- ☐ Memory Governance Center
- ☐ Human Validation Console
- ☐ Process Validation Console
- ☐ Session Ledger

Ordre d'execution :

8. ☐ Creer modeles memory
   - Fichier ou zone : `workcenter/schemas/memory_schema.json`
   - Pourquoi : Normaliser les statuts memoire
   - Critere : test PASS, UI charge, aucune action interdite.
9. ☐ Creer Memory Governance UI
   - Fichier ou zone : `MemoryGovernancePage.tsx`

- Pourquoi : Visualiser candidates et promotions
- Critere : test PASS, UI charge, aucune action interdite.

10. ☐ Creer Human Validation Queue
    - Fichier ou zone : `HumanValidationPage.tsx`
    - Pourquoi : Eviter promotion auto
    - Critere : test PASS, UI charge, aucune action interdite.
11. ☐ Creer Process Validation Queue
    - Fichier ou zone : `ProcessValidationPage.tsx`
    - Pourquoi : Verifier par test/runtime/freeze/code
    - Critere : test PASS, UI charge, aucune action interdite.
12. ☐ Creer Session close dry-run
    - Fichier ou zone : `POST /api/workcenter/session/close/dry-run`
    - Pourquoi : Preparer ledger sans ecriture dangereuse
    - Critere : test PASS, UI charge, aucune action interdite.
13. ☐ Tests anti auto promotion
    - Fichier ou zone : `tests/test_memory_no_auto_promotion.py`
    - Pourquoi : Prouver le verrou
    - Critere : test PASS, UI charge, aucune action interdite.

Gate de sortie :

- ☐ Toutes les pages V2 affichent leurs donnees ou un etat vide clair.
- ☐ Tous les tests V2 passent.
- ☐ Le rapport d'acceptance V2 est ecrit.
- ☐ Aucune mutation directe du Kernel, de Graphiti ou du canon n'est possible depuis V2.

7.3 V3 - Proof Freeze Education

Role : Relier preuve, replay, freeze candidate et education Brody.

Pages concernees :

- ☐ Proof and Replay Lab
- ☐ Freeze Factory
- ☐ Brody Education Station
- ☐ Resource Library

Ordre d'execution :

14. ☐ Brancher Proof Registry
    - Fichier ou zone : `proofs_registry.json`
    - Pourquoi : Afficher proofs et statuts

- Critere : test PASS, UI charge, aucune action interdite.

15. ☐ Creer Proof Lab UI
    - Fichier ou zone : `ProofReplayLabPage.tsx`
    - Pourquoi : Lire proofs et run requests
    - Critere : test PASS, UI charge, aucune action interdite.
16. ☐ Creer Replay readonly
    - Fichier ou zone : `GET /api/workcenter/replay/{decision_id}`
    - Pourquoi : Rejouer sans muter
    - Critere : test PASS, UI charge, aucune action interdite.
17. ☐ Creer Freeze candidates
    - Fichier ou zone : `FreezeFactoryPage.tsx`
    - Pourquoi : Preparer freeze evidence
    - Critere : test PASS, UI charge, aucune action interdite.
18. ☐ Creer Brody Education registry
    - Fichier ou zone : `brody_training_registry.json`
    - Pourquoi : Lister modules T1-T20
    - Critere : test PASS, UI charge, aucune action interdite.
19. ☐ Creer Brody Education UI
    - Fichier ou zone : `BrodyEducationPage.tsx`
    - Pourquoi : Apprendre sans souverainete
    - Critere : test PASS, UI charge, aucune action interdite.

Gate de sortie :

- ☐ Toutes les pages V3 affichent leurs donnees ou un etat vide clair.
- ☐ Tous les tests V3 passent.
- ☐ Le rapport d'acceptance V3 est ecrit.
- ☐ Aucune mutation directe du Kernel, de Graphiti ou du canon n'est possible depuis V3.

7.4 V4 - Terrain Sandbox Multimodal

Role : Ajouter les surfaces avancees en mode controle et trace.

Pages concernees :

- ☐ Terrain Branching Center
- ☐ Coding Sandbox
- ☐ Server Control Advisory
- ☐ Multimodal Workbench

Ordre d'execution :

20. ☐ Creer Terrain Branching registry
    - Fichier ou zone : `domains_registry.json + terrain registry`
    - Pourquoi : Organiser experimentations
    - Critere : test PASS, UI charge, aucune action interdite.
21. ☐ Creer Coding Sandbox readonly
    - Fichier ou zone : `CodingSandboxPage.tsx`
    - Pourquoi : Lire et tester sans commit direct
    - Critere : test PASS, UI charge, aucune action interdite.
22. ☐ Creer Kernel Change Protocol UI
    - Fichier ou zone : `kernel_change_registry.json`
    - Pourquoi : Gerer evolution kernel sans mutation directe
    - Critere : test PASS, UI charge, aucune action interdite.
23. ☐ Creer Multimodal candidate intake
    - Fichier ou zone : `MultimodalWorkbenchPage.tsx`
    - Pourquoi : Classer medias en candidates
    - Critere : test PASS, UI charge, aucune action interdite.
24. ☐ Ajouter event log
    - Fichier ou zone : `workcenter_event_log`
    - Pourquoi : Tracer actions cockpit
    - Critere : test PASS, UI charge, aucune action interdite.
25. ☐ Ajouter observabilite avancee
    - Fichier ou zone : `OTEL/Langfuse/Grafana later`
    - Pourquoi : Cible maximale
    - Critere : test PASS, UI charge, aucune action interdite.

Gate de sortie :

- ☐ Toutes les pages V4 affichent leurs donnees ou un etat vide clair.
- ☐ Tous les tests V4 passent.
- ☐ Le rapport d'acceptance V4 est ecrit.
- ☐ Aucune mutation directe du Kernel, de Graphiti ou du canon n'est possible depuis V4.

7.5 V5 - Final Maximal

Role : Durcir, industrialiser et rendre le WorkCenter stable sans devoir revenir sur le socle.

Pages concernees :

- ☐ Toutes les pages

Ordre d'execution :

26. ☐ Migrer registries critiques vers DB
    - Fichier ou zone : `database migrations`
    - Pourquoi : Stabiliser multi-user et historique
    - Critere : test PASS, UI charge, aucune action interdite.
27. ☐ Ajouter permissions par role
    - Fichier ou zone : `RBAC service`
    - Pourquoi : Securiser usages avancees
    - Critere : test PASS, UI charge, aucune action interdite.
28. ☐ Ajouter audit export
    - Fichier ou zone : `audit export service`
    - Pourquoi : Preparer institutionnel
    - Critere : test PASS, UI charge, aucune action interdite.
29. ☐ Ajouter workers async
    - Fichier ou zone : `proof/freeze/replay workers`
    - Pourquoi : Eviter timeouts
    - Critere : test PASS, UI charge, aucune action interdite.
30. ☐ Ajouter tests e2e complets
    - Fichier ou zone : `e2e/workcenter/*.spec.ts`
    - Pourquoi : Prouver les parcours
    - Critere : test PASS, UI charge, aucune action interdite.
31. ☐ Freeze WorkCenter final
    - Fichier ou zone : `WORKCENTER_FREEZE`
    - Pourquoi : Sceller la version stable
    - Critere : test PASS, UI charge, aucune action interdite.

Gate de sortie :

- ☐ Toutes les pages V5 affichent leurs donnees ou un etat vide clair.
- ☐ Tous les tests V5 passent.
- ☐ Le rapport d'acceptance V5 est ecrit.
- ☐ Aucune mutation directe du Kernel, de Graphiti ou du canon n'est possible depuis V5.

8. Routes API WorkCenter cible

8.1 Routes GET readonly

[ ] GET /api/workcenter/status

[ ] GET /api/workcenter/metrics

[ ] GET /api/workcenter/components

[ ] GET /api/workcenter/servers

[ ] GET /api/workcenter/routes

[ ] GET /api/workcenter/domains

[ ] GET /api/workcenter/authority

[ ] GET /api/workcenter/memory/candidates

[ ] GET /api/workcenter/memory/promoted

[ ] GET /api/workcenter/validation/human

[ ] GET /api/workcenter/validation/process

[ ] GET /api/workcenter/training/modules

[ ] GET /api/workcenter/training/modules/{module_id}

[ ] GET /api/workcenter/terrain/domains

[ ] GET /api/workcenter/terrain/branches

[ ] GET /api/workcenter/proofs

[ ] GET /api/workcenter/replay/{decision_id}

[ ] GET /api/workcenter/freezes

[ ] GET /api/workcenter/resources

[ ] GET /api/workcenter/resources/{resource_id}

[ ] GET /api/workcenter/session/current

[ ] GET /api/workcenter/sessions

[ ] GET /api/workcenter/sandbox/status

[ ] GET /api/workcenter/servers/commands/advisory

[ ] GET /api/workcenter/multimodal/resources

[ ] GET /api/workcenter/kernel-change/protocol

[ ] GET /api/workcenter/audit/events

8.2 Routes POST limitees a demandes, dry-run ou proposals

[ ] POST /api/workcenter/session/close/dry-run

[ ] POST /api/workcenter/memory/candidate

[ ] POST /api/workcenter/validation/request

[ ] POST /api/workcenter/validation/human/approve

[ ] POST /api/workcenter/validation/human/reject

[ ] POST /api/workcenter/validation/process/request

[ ] POST /api/workcenter/training/answer

[ ] POST /api/workcenter/terrain/branch/request

[ ] POST /api/workcenter/proofs/run/request

[ ] POST /api/workcenter/freezes/candidate

[ ] POST /api/workcenter/freezes/run/request

[ ] POST /api/workcenter/sandbox/patch/propose

[ ] POST /api/workcenter/sandbox/test/request

[ ] POST /api/workcenter/multimodal/candidate

[ ] POST /api/workcenter/kernel-change/request

8.3 Routes explicitement interdites

[ ] POST /api/workcenter/kernel/mutate

[ ] POST /api/workcenter/kernel/overwrite

[ ] POST /api/workcenter/graphiti/write-direct

[ ] POST /api/workcenter/memory/promote-canon-direct

[ ] POST /api/workcenter/freezes/force-without-validation

[ ] POST /api/workcenter/act/real-action

[ ] POST /api/workcenter/git/commit-direct

[ ] POST /api/workcenter/git/merge-direct

9. Protocoles speciaux

9.1 Kernel Change Protocol

Ce protocole existe pour corriger la confusion sur la mutation Kernel. Le Kernel peut evoluer, mais pas par action directe du cockpit.

[ ] Une demande Kernel change possede un ID.

[ ] Elle explique le besoin.

[ ] Elle liste les fichiers potentiellement touches.

[ ] Elle liste les invariants impactes.

[ ] Elle liste les tests a ajouter ou modifier.

[ ] Elle liste les preuves a relancer.

[ ] Elle exige une branche dediee.

[ ] Elle exige un audit local.

[ ] Elle exige un freeze pre-merge.

[ ] Elle exige validation humaine.

[ ] Elle interdit modification depuis UI directe.

[ ] Elle interdit merge direct.

9.2 Memory Promotion Protocol

[ ] Toute memoire durable commence en candidate.

[ ] Toute promotion training exige validation ou process.

[ ] Toute promotion canon exige validation humaine.

[ ] Toute promotion proof exige artefact proof.

[ ] Toute promotion operational exige source, date, statut, risque.

[ ] Toute memoire rejected reste visible dans l'historique.

[ ] Toute memoire forbidden_secret est bloquee et masquee selon politique.

9.3 Freeze Protocol

[ ] Freeze candidate creee.

[ ] Scope freeze liste.

[ ] Evidence collectee.

[ ] Manifest SHA256 genere.

[ ] Smoke test execute.

[ ] Rapport freeze ecrit.

[ ] Ledger mis a jour.

[ ] Freeze visible dans Freeze Factory.

[ ] Aucun freeze force sans evidence.

10. Criteres d'acceptation globaux

[ ] Le cockpit affiche l'etat reel au lieu d'inventer.

[ ] Le cockpit distingue runtime, test, freeze, code, documented only, legacy, unknown.

[ ] Chaque page a un etat loading.

[ ] Chaque page a un etat empty.

[ ] Chaque page a un etat error.

[ ] Chaque page a des badges de statut canoniques.

[ ] Chaque action sensible cree une request, pas une mutation directe.

[ ] Chaque request possede actor, timestamp, reason, before, after, evidence.

[ ] Brody reste non souverain.

[ ] Graphiti reste non souverain.

[ ] CIC reste contexte readonly.

[ ] Sigma reste aggregation et evaluation, pas decision souveraine.

[ ] Kernel reste seule autorite de decision.

[ ] Le Kernel peut evoluer uniquement via Kernel Change Protocol.

[ ] La memoire canonique exige validation humaine.

[ ] La memoire proof exige artefact verifiable.

[ ] Les proofs ne sont pas claim confirmed sans run, freeze, code ou audit clair.

[ ] Les freezes exigent manifest et rapport.

[ ] Le sandbox ne commit pas directement.

[ ] Le Server Advisory ne kill/restart pas directement.

[ ] Le Multimodal Workbench ne cree pas memoire durable directement.

[ ] Tous les tests WorkCenter passent.

[ ] Un rapport WORKCENTER_ACCEPTANCE_REPORT.md est produit.

11. Definition of Done finale

Le support d'implementation est considere complet quand :

[ ] Toutes les pages du cahier des charges ont une fiche execution.

[ ] Chaque fiche relie page, fichier, route, test, critere et interdit.

[ ] Les versions V1, V2, V3, V4 et V5 sont separees.

[ ] La cible finale maximale est conservee.

[ ] Le socle minimal ne devient pas une impasse.

[ ] La nuance Kernel est corrigee : pas de mutation directe, mais evolution possible via protocole.

[ ] Le developpeur peut commencer V0 sans poser de question de structure.

[ ] Le developpeur peut commencer V1 apres validation V0.

[ ] Le cockpit est relie au cahier des charges, pas invente a cote.

[ ] Rien dans le document ne pousse a build direct sans validation.